

Gradient Descent for Linear Regression

By combining the gradient descent algorithm with the specific squared error cost function for linear regression, we get a complete algorithm for training the model.

The Complete Algorithm

The final algorithm involves repeatedly performing the following **simultaneous updates** for w and b until the model converges.

- **Update for w :**

$$w := w - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)} \right]$$

- **Update for b :**

$$b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \right]$$

Key Components:

- $f_{w,b}(x^{(i)})$ is the model's prediction, calculated as $w x^{(i)} + b$.
 - $(f_{w,b}(x^{(i)}) - y^{(i)})$ is the error for the i -th training example.
-

The Derivative Terms (Calculus Derivation Optional)

The specific formulas in the update rules come from calculating the partial derivatives of the squared error cost function, $J(w, b)$.

- **Derivative with respect to w :**

$$\frac{\partial}{\partial w} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

- **Derivative with respect to b :**

$$\frac{\partial}{\partial b} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

You don't need to know calculus to implement the algorithm; you only need to use these final formulas.

A Special Property: Convexity

The squared error cost function used for linear regression has a special and very useful property: it is a **convex function**.

- **What this means:** The cost function has a single **bowl shape** and therefore has **no local minima**, only **one global minimum**.
- **Why it's important:** Because the cost function is convex, gradient descent is **guaranteed**

to converge to the global minimum (the best possible solution), as long as the learning rate α is chosen appropriately. It will never get "stuck" in a suboptimal valley.